CO4AT1
CSA0918 - JAVA PROGRAMMING
NAME: J O JOANNA DIVINE
REG NO: 192572003

Java Industry Problem-Solving: Elevated Permissions for Sandboxed Applet File Access

---

1. Title

Breaking Out of the Applet Sandbox Safely: Code Signing for Local File Access in a Batch Export Feature

2. Problem Statement

A client's internal applet runs inside the browser's security sandbox, as all applets do by default. A new batch export feature requires the applet to write temporary calculation files to the user's local disk. Security auditors flag that the sandbox forbids this local filesystem access by default.

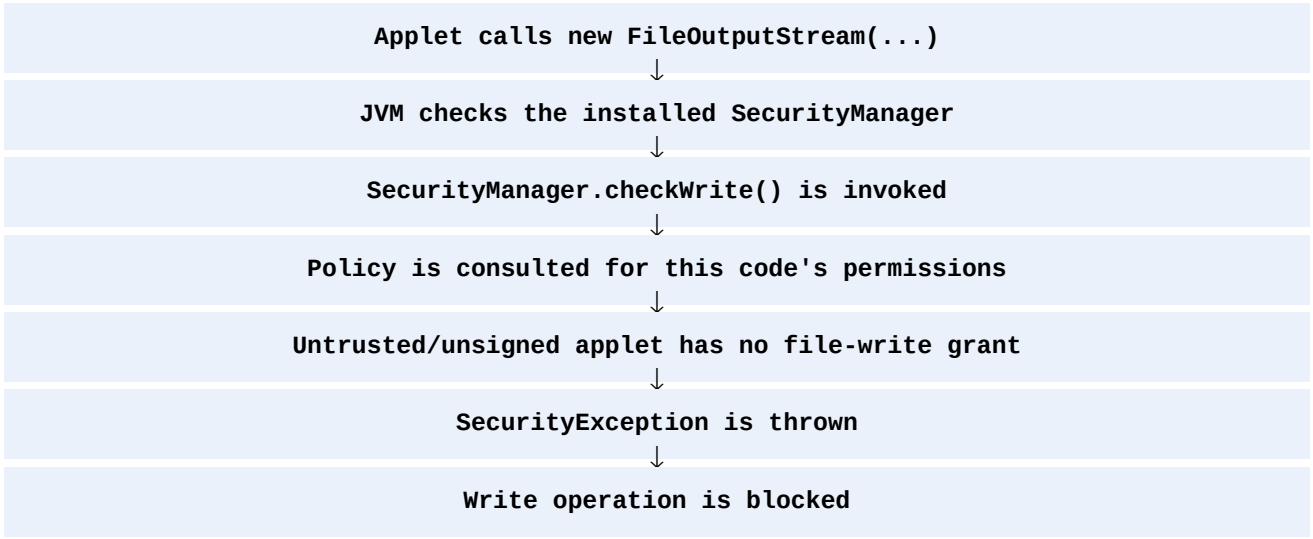
The architectural reason applets cannot freely touch the local filesystem must be explained, and the standard industry mechanism used to obtain elevated permission for a specific, legitimate need such as this must be described.

3. Problem Diagnosis

An applet is downloaded from a remote server and executed inside the user's browser on the user's own machine. Because the code originates from an untrusted, external source, the Java Virtual Machine does not extend it the same trust it gives to code that is installed locally.

What enforces this restriction?

Every security-sensitive operation an applet attempts, such as opening a local file, is intercepted by a component called the SecurityManager. Before the operation is allowed to proceed, the SecurityManager consults the currently installed security policy.



By default, code loaded from a remote codebase runs inside the restricted sandbox and is granted only a minimal set of permissions: it can paint to its own applet panel, make network connections back to its own originating server, and little else. Reading or writing arbitrary paths on the local disk is excluded from that default grant.

4. Role of the Sandbox and SecurityManager

SecurityManager

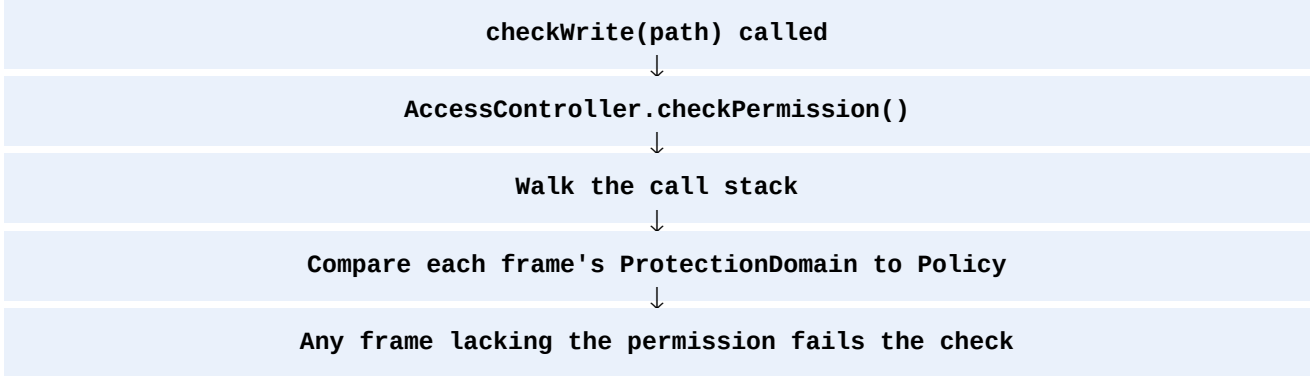
SecurityManager is the gatekeeper class that every potentially dangerous API call passes through. It does not decide permissions itself; it asks the policy.

```
public void checkWrite(String file) {
    // consults AccessController / Policy before allowing the write
}
```

AccessController and Policy

The AccessController walks the call stack and checks each frame's protection domain against the active Policy object. The protection domain of a class is determined by where the class came from (its codebase URL) and, critically, whether it was cryptographically signed and by whom.

Therefore:



5. Architectural Root Cause

The primary reason applets cannot freely access the local filesystem is that applet code is remote, mobile code executing with the user's own local privileges inside the user's browser process.

If unrestricted file access were granted by default:

- A malicious or compromised applet could silently read private documents from the user's disk.
- It could overwrite or delete arbitrary files, including system or configuration files.
- It could plant executable files that run outside the sandbox entirely.

Root cause

Trust cannot be inferred from network delivery alone. Because the JVM cannot verify the intent or safety of remotely loaded, unsigned code, the sandbox denies filesystem access by default as a conservative, fail-safe design decision, and only relaxes that restriction when the code's origin and integrity can be verified.

6. Industry Solution — Code Signing and Explicit Policy Grants

The standard mechanism is to digitally sign the applet's JAR file with a certificate from a trusted Certificate Authority, and to grant that specific signed code an explicit, narrow permission through the security policy, rather than disabling the sandbox altogether.

Instead of the applet running as anonymous, untrusted code, the browser's plug-in verifies the signature, confirms the publisher's identity, and (after user consent for self-signed or unverified certificates) elevates that code's protection domain to include the permissions the policy or manifest defines for it — typically the minimum needed, such as write access to one temporary export directory rather than the whole disk.

Without elevated permission

```
Unsigned applet JAR
        ↓
Loaded into restricted sandbox
        ↓
checkWrite() denied by default policy
        ↓
SecurityException
```

With code signing

```
Developer signs JAR with jarsigner + CA certificate
        ↓
Browser plug-in verifies signature and publisher
        ↓
User is prompted to trust the publisher
        ↓
Signed code's ProtectionDomain includes granted permission
        ↓
checkWrite() to the export path succeeds
```

The user, not the applet, ultimately decides whether to trust the signed publisher; the mechanism moves the trust decision to an identifiable party instead of granting blanket access to anonymous code.

7. Algorithm

1. Package the applet classes into a JAR file.
2. Generate or obtain a code-signing certificate from a trusted Certificate Authority.
3. Sign the JAR using jarsigner with that certificate.
4. Deploy the signed JAR and reference it from the applet tag or JNLP descriptor.
5. On load, the plug-in verifies the signature and prompts the user to accept the publisher's certificate.
6. Grant only the specific permission needed (for example, write access to one export directory) via the policy file or signed-JAR permission entry.
7. Wrap the sensitive file operation in a doPrivileged() block so the elevated permission applies only to that code path.
8. Perform the batch export write; all other operations remain confined to the sandbox.

8. Java Implementation

Note: Java Applets and the classic browser plug-in model are obsolete/deprecated in modern Java environments. The following demonstrates the signed-applet and doPrivileged() permission-elevation concept requested by the problem.

```
import java.applet.Applet;
import java.io.*;
import java.security.*;

public class BatchExportApplet extends Applet {

    public void exportCalculations(final String data) {
        try {
            // Elevate only this block to use the signed JAR's granted
            // file-write permission; nothing else in the applet gets it.
            AccessController.doPrivileged(new PrivilegedExceptionAction<Void>() {
                public Void run() throws IOException {
                    File tempFile = File.createTempFile("export_", ".tmp");
                    try (FileWriter writer = new FileWriter(tempFile)) {
                        writer.write(data);
                    }
                    return null;
                }
            });
        } catch (PrivilegedActionException e) {
            // Unsigned or untrusted context: permission was never granted
            e.printStackTrace();
        }
    }
}
```

Corresponding policy grant (java.policy)

```
grant signedBy "TrustedExportPublisher" {
    permission java.io.FilePermission
        "\${user.home}\${/}export\${/}*", "write";
```

```
};
```

Signing command

```
jarsigner -keystore export.keystore BatchExportApplet.jar TrustedExportPublisher
```

9. Important Part of the Code

The key part is:

```
AccessController.doPrivileged(new PrivilegedExceptionAction<Void>() { ... });
```

This narrows the elevated trust to exactly the block that performs the file write. Instead of the whole applet running with elevated rights, only the code inside doPrivileged() is permitted to exercise the grant defined for the signed publisher.

The policy entry restricts that grant further, to a single directory pattern (user.home/export/*) rather than the entire filesystem, and it is scoped to code signed by a named, verifiable publisher.

10. Why the Default Policy Denies the Write

An unsigned applet's protection domain carries no FilePermission grant, so any attempt to construct a FileOutputStream or FileWriter on the local disk fails the AccessController's stack check before a single byte is written.

That failure is intentional: it forces every path to local-disk access through an explicit, auditable grant tied to a verified signer, rather than allowing silent, ambient access to any code the browser happens to download.

11. Working of the Permission-Elevation Mechanism

The complete process is:

```
Applet JAR built and signed with publisher certificate
                        ↓
Browser loads JAR, verifies signature chain
                        ↓
User accepts the publisher's certificate
                        ↓
Signed code assigned an elevated ProtectionDomain
                        ↓
doPrivileged() block executes file write
                        ↓
AccessController finds matching grant in policy
                        ↓
checkWrite() succeeds; temp export file created
```

The important idea is:

Verify identity first, grant the narrowest permission second.

12. Comparison

Unsigned / Sandboxed Applet	Signed Applet with Explicit Grant
No local file access by default	Local file access limited to a granted path
Runs as fully untrusted code	Runs with an identifiable publisher's trust
Any file operation throws SecurityException	Only permitted operations inside doPrivileged() succeed
Protects users from unknown/malicious code	Protects users while enabling legitimate features
No certificate or user consent involved	Requires CA-issued certificate and user acceptance
Zero configuration needed	Requires signing, policy, and deployment setup

13. Industry-Level Approach

For a real modern desktop deployment, the same trust-then-grant principle is commonly handled by the deployment framework rather than a raw, manually signed Applet. Java Web Start applications used a JNLP descriptor with a <security><all-permissions/></security> element backed by the same signed-JAR verification, and modern Java desktop apps (JavaFX, or plain installed JARs) simply run with full local trust because they are installed locally rather than fetched anonymously from a browser each time.

```
<jnlp>
  <security>
    <all-permissions/>
  </security>
  <resources>
    <jar href="BatchExportApplet.jar" />
  </resources>
</jnlp>
```

A modern web application facing this same requirement today would typically avoid client-side file trust elevation entirely and instead generate the export server-side, streaming the completed file to the browser's normal, sandboxed download mechanism — which needs no elevated permission at all.

14. Expected Result

After implementing signed-JAR permission elevation:

1. The applet remains sandboxed for every operation outside the granted scope.
2. The user is shown a verifiable publisher identity before any elevated trust is granted.
3. Only the doPrivileged() export block can write to the granted temporary export path.
4. SecurityException is no longer thrown for the batch export feature.
5. The rest of the applet's attack surface is unchanged and remains fully restricted.

15. Conclusion

Applets cannot freely access the local filesystem because they are remote, mobile code executing with the user's local process privileges, and the JVM has no way to verify the safety or intent of anonymous, unsigned code by default; the SecurityManager and its underlying Policy therefore deny filesystem access unless an explicit grant exists.

The industry-standard mechanism to obtain elevated permission for a specific, legitimate need is to digitally sign the applet's JAR with a trusted certificate, have the user accept that publisher's identity, define the narrowest possible permission grant in the security policy, and confine the sensitive operation inside an AccessController.doPrivileged() block. This converts a blanket trust decision into an identifiable, auditable, and minimally scoped one.

Final Answer in One Line

Applets are denied local file access by default because unsigned, remotely loaded code cannot be trusted by the JVM's SecurityManager; the industry standard is to digitally sign the JAR, have the user accept the verified publisher, grant the narrowest possible permission in the security policy, and confine the file operation inside AccessController.doPrivileged() so elevated trust applies only to that specific, audited need.